

# | Quality Assurance in SW-Development

## Assignment 1 "Tutorial": JaCoCo and Pitest

Sebastian David Albu

Summer Semester 2026

Institute of Software Engineering and Artificial Intelligence

LaTeX Template provided by the Institute of Information Security

# Motivation

## “The Philosophy

*"Don't write test cases,  
generate them!"*

– John Hughes

### But... Why?

- Why automate?
- Why not just write them manually?

*You will see the answer after this assignment!*



# Your Task

## </> The System

You are given a Java project:

- **Actor System**
- **Message Board**

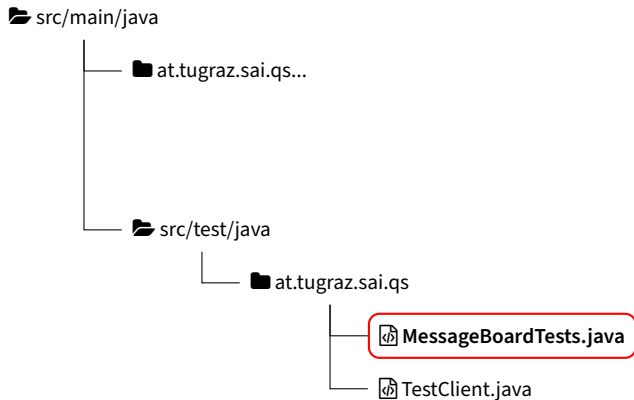
(Details discussed at the end)

## 🏆 The Goal (Group Work)

- 1 **100% Line Coverage**
- 2 **100% Branch Coverage**
- 3  **$\geq 90\%$  Mutation Coverage**

💡 **How?** By implementing robust test cases in `MessageBoardTests.java`!

# Test Cases Setup



# Test Cases Setup



## Implementation

- Implement **ALL** tests in `MessageBoardTests.java`.
- Run them using the **Gradle** interface (built-in in IntelliJ).

## Requirements

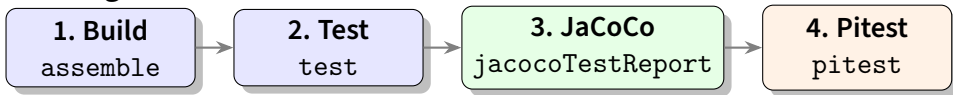
- All tests must **succeed**.
- Assertions must be **meaningful!**
- **No minimum number** of tests, but coverage goals apply.

# Gradle Workflow

## Importing the Project:

- 1 Extract .zip → Right-click → Open as IntelliJ Project.
- 2 Ensure **JDK 17** is selected.
- 3 Accept the Gradle configuration popup.

## Running Tasks:



# JaCoCo: Code & Branch Coverage



## Q Finding the Report

`<Project>/MessageBoard/build/reports/jacoco/test/html/index.html`

## Understanding the Colors:

**GREEN**

Full

Coverage  
(All branches hit)

**YELLOW**

Partial

Coverage  
(Some branches missed)

**RED**

No

Coverage  
(Never reached)

🖱️ *Hover over the yellow diamond icons in the report to see exactly how many branches were missed!*

# Pitest: Mutation Coverage

## 🔍 Finding the Report

`<Project>/MessageBoard/build/reports/pitest/test/html/index.html`

## 🛠️ How it works

Pitest modifies ("mutates") your **bytecode**:

- Changes `>=` to `<`
- Removes method calls
- Changes return values

## 🛡️ Your Goal

**Kill the Mutants!**

- If a test fails → **KILLED** (Good!)
- If all tests pass → **SURVIVED** (Bad!)

# Pitest: Mutation Coverage



## Example Mutation:

22 1. replaced int return with 0 ... -> KILLED

*(Means your test successfully noticed the return value changed)*

# Structure & Strategy

## Next Steps:

### 1 Analyze the Code Base:

- Compare code with the assignment PDF.
- Understand the ActorSystem structure.
- *Note: A similar framework is used for Assignment 2!*

### 2 Plan Tests:

- Don't just code blindly.
- Think about edge cases first.

### 3 Iterate:

- Write Test → Check Coverage → Improve.

Happy  
Testing!



# Hints for the First Partial Exam

## ✓ Coverage (JaCoCo)

### Understand the mechanics:

- **Line Coverage:** Did the execution hit this line of code?
- **Branch Coverage:** Did we take *both* the true and false paths of an if?

## 🔧 Mutation Testing (Pitest)

### Know how it works internally:

- How code is mutated (bytecode modification).
- **Killed:** Test fails (Good).
- **Survived:** Test passes (Bad).

# Hints for the First Partial Exam

## Theory: Boundary Value Analysis

*"Don't forget the Lecture material!"*

Boundary Value Analysis is **not** explicitly forced in the assignment, but it is **vital** for the exam. (Maybe you can think about ways to integrate them already into this assignment)

**Concept:** Errors usually hide at the edges.

- If input is 1 . . 10:
- Test: 0, 1, 5, 10, 11.

Questions?

Live-Demo :)

Any final Questions?

# | Quality Assurance in SW-Development

## Assignment 1 "Tutorial": JaCoCo and Pitest

Sebastian David Albu

Summer Semester 2026

Institute of Software Engineering and Artificial Intelligence

LaTeX Template provided by the Institute of Information Security